

Invoice Generation API Documentation

Version: 1.0.0

Base URL: `http://your-server:8000`

Last Updated: November 24, 2025

Table of Contents

1. [Overview](#)
 2. [Authentication](#)
 3. [Common Response Format](#)
 4. [Tax Invoice Endpoints](#)
 5. [Bill Invoice Endpoints](#)
 6. [Error Handling](#)
 7. [Data Models](#)
 8. [Examples](#)
 9. [Testing](#)
-

Overview

This API provides AI-powered invoice generation capabilities for two types of invoices:

1. **Tax Invoice** (Siva Sakthi GTA) - Standard tax invoices with items, GST calculations
2. **Bill Invoice** (Freight Bill) - Transportation/freight bills with delivery runs

Key Features

-  **AI-Powered Data Extraction** - Upload Excel/CSV files and extract data automatically

-  **Natural Language Commands** - Use plain English to modify invoices
-  **Automatic Calculations** - GST, totals, and line items calculated automatically
-  **Export to PDF/HTML** - Generate professional invoices in multiple formats
-  **Stateless API** - No session management required, send full state with each request

Authentication

Current Version: No authentication required (development mode)

Production: Add your authentication headers here when deploying to production.

Common Response Format

All processing endpoints return a consistent response format:

```
{
  "success": true,
  "state": { /* Invoice state object */ },
  "message": "Operation completed successfully",
  "timestamp": "2025-11-24T17:00:00.000000"
}
```

Fields

Field	Type	Description
success	boolean	Whether the operation succeeded
state	object	Complete invoice state (see Data Models)
message	string	Human-readable status message
timestamp	string	ISO 8601 timestamp of the response

Tax Invoice Endpoints

1. Get Default Tax Invoice State

Endpoint: GET /api/tax/default

Description: Get a new tax invoice with all fields initialized to default values.

Response:

```
{
  "success": true,
  "state": {
    "invoice_number": "",
    "invoice_date": "",
    "reverse_charge": "YES",
    "supply_mode": "Goods Transport Agency Service",
    "company_info": { /* Company details */ },
    "billing_to": { /* Client details */ },
    "items": [],
    "cgst_rate": 6.0,
    "sgst_rate": 6.0,
    "iqst_rate": 5.0,
    "subtotal": 0.0,
    "total": 0.0
  },
  "message": "Default tax invoice state",
  "timestamp": "2025-11-24T17:00:00.000000"
}
```

2. Process Tax Invoice

Endpoint: POST /api/tax/process

Description: Process a tax invoice with optional file upload and natural language command.

Content-Type: multipart/form-data

Parameters:

Parameter	Type	Required	Description
file	File	No	Excel/CSV file to extract data from
state	JSON string	Yes	Current invoice state
command	string	Yes	Natural language command

Example Request (with file):

```
curl -X POST http://localhost:8000/api/tax/process \
-F "file=@invoice_data.xlsx" \
-F "state={\"invoice_number\": \"\", \"items\": [], ...}" \
-F "command=add all details from dataset"
```

Example Request (without file):

```
curl -X POST http://localhost:8000/api/tax/process \
-F "state={\"invoice_number\": \"INV-001\", \"items\": [...], ...}" \
-F "command=set CGST rate to 9%"
```

Supported Commands:

With File Upload: - "add all details from dataset" - Extract and add all rows - "add rows from September 2025" - Filter by date - "add rows where quantity > 100" - Filter by condition

Without File: - "set invoice number to INV-001" - Update header fields - "set CGST rate to 9%" - Update tax rates - "delete row 3" - Remove specific item - "change quantity in row 2 to 50" - Update item field

Response:

```
{
  "success": true,
  "state": { /* Updated invoice state */ },
  "message": "✓ Agenic operation complete: ADD",
  "timestamp": "2025-11-24T17:00:00.000000"
}
```

3. Export Tax Invoice

Endpoint: POST /api/tax/export

Description: Export tax invoice to PDF or HTML format.

Content-Type: application/json

Request Body:

```
{
  "state": { /* Invoice state */ },
  "format": "pdf", // or "html"
  "template_name": "Siva Sakthi GTA" // optional
}
```

Response (PDF): - Content-Type: application/pdf - Returns PDF file as binary data

Response (HTML):

```
{
  "html": "<html>...</html>"
}
```

Example:

```
curl -X POST http://localhost:8000/api/tax/export \
  -H "Content-Type: application/json" \
  -d '{
    "state": {...},
    "format": "pdf"
  }' \
  --output invoice.pdf
```

Bill Invoice Endpoints

1. Get Default Bill Invoice State

Endpoint: GET /api/bill/default

Description: Get a new bill invoice (freight bill) with all fields initialized.

Response:

```

{
  "success": true,
  "state": {
    "freight_bill": {
      "series_no": "",
      "bill_date": "",
      "po_no": "",
      "pan_no": "ARMPS3396J",
      "summary": "",
      "to_party": {
        "name": "",
        "address": ""
      },
      "runs": [],
      "dc_total": 0.0,
      "gr_total": 0.0,
      "subtotal": 0.0,
      "total": 0.0
    },
    "message": "Default bill invoice state",
    "timestamp": "2025-11-24T17:00:00.000000"
  }
}

```

2. Process Bill Invoice

Endpoint: POST /api/bill/process

Description: Process a bill invoice with optional file upload and natural language command.

Content-Type: multipart/form-data

Parameters:

Parameter	Type	Required	Description
file	File	No	Excel/CSV file to extract delivery runs from
state	JSON string	Yes	Current invoice state
command	string	Yes	Natural language command

Example Request:

```

curl -X POST http://localhost:8000/api/bill/process \
-F "file=@freight_data.xlsx" \
-F "state={\"freight_bill\":{\"runs\":[],...}}\" \
-F "command=add all details from dataset to invoice"

```

Supported Commands:

With File Upload: - "add all details from dataset to invoice" - Extract all delivery runs - "add rows from October 2025" - Filter by date - "add rows where truck number starts with TN32" - Filter by condition

Without File: - "fill all empty LR No with values 40, 42, 44" - Fill empty fields - "set rate to 250 for all rows" - Bulk update - "delete all rows where truck no ends with 66" - Conditional delete - "change truck number in row 1 to TN99Z9999" - Update specific row

Response:

```
{
  "success": true,
  "state": {
    "freight_bill": {
      "series_no": "",
      "runs": [
        {
          "date": "21-10-2025",
          "truck_no": "AP39TU3699",
          "lr_no": "",
          "dc_qty_mt": 34.72,
          "gr_qty_mt": 34.44,
          "rate": 710,
          "line_total": 24452.4
        }
      ],
      "total": 24452.4
    }
  },
  "message": "✓ Agentic operation complete: ADD",
  "timestamp": "2025-11-24T17:00:00.000000"
}
```

3. Export Bill Invoice

Endpoint: POST /api/bill/export

Description: Export bill invoice to PDF or HTML format.

Content-Type: application/json

Request Body:

```
{
  "state": { /* Invoice state */ },
  "format": "pdf", // or "html"
  "template_name": "Siva Sakthi Freight Bill" // optional
}
```

Response: Same as Tax Invoice export endpoint.

Error Handling

Error Response Format

```
{
  "detail": "Error message describing what went wrong"
}
```

Common HTTP Status Codes

Code	Meaning	Description
200	OK	Request succeeded
400	Bad Request	Invalid JSON or missing required fields
500	Internal Server Error	Processing failed (check error message)

Example Error Response

```
{
  "detail": "Processing failed: Invalid JSON in state parameter"
}
```

Data Models

Tax Invoice State

```
interface TaxInvoiceState {
    invoice_number: string;
    invoice_date: string;
    reverse_charge: "YES" | "NO";
    supply_mode: string;
    supply_period: string;
    place_of_supply: string;

    company_info: {
        name: string;
        taqline: string;
        address: string;
        phone: string;
        mobile: string;
        email: string;
        gstin: string;
        pan: string;
        state: string;
        state_code: string;
        bank: {
            account_no: string;
            name: string;
            ifsc: string;
            branch: string;
        };
    };

    billing_to: {
        client_name: string;
        address: string;
        gstin: string;
        state: string;
        state code: string;
        purchase_order_no: string;
        spo_no: string;
    };

    items: Array<{
        description: string;
        hsn_code: string;
        uom: string;
        quantity: number;
        unit price: number;
        line_total: number;
    }>;

    cgst_rate: number;
    sgst_rate: number;
    igst_rate: number;
    subtotal: number;
    cgst amount: number;
    sgst_amount: number;
    igst_amount: number;
    total: number;
}
```

Bill Invoice State

```
interface BillInvoiceState {
  freight_bill: {
    series_no: string;
    bill_date: string;
    po_no: string;
    pan_no: string;
    summary: string;

    to_party: {
      name: string;
      address: string;
    };

    runs: Array<{
      date: string;           // Format: DD-MM-YYYY
      truck_no: string;
      lr_no: string;
      dc_qty_mt: number;      // DC Quantity in MT
      gr_qty_mt: number;      // GR Quantity in MT
      rate: number;           // Rate per MT
      line_total: number;     // Calculated: gr_qty_mt × rate
    }>;

    dc_total: number;         // Sum of all dc_qty_mt
    gr_total: number;         // Sum of all gr_qty_mt
    subtotal: number;         // Sum of all line_total
    total: number;            // Final total
  };
}
```

Examples

Complete Workflow: Bill Invoice

Step 1: Get Default State

```
const response = await fetch('http://localhost:8000/api/bill/default');
const { state } = await response.json();
```

Step 2: Upload File and Extract Data

```
const formData = new FormData();
formData.append('file', fileInput.files[0]);
formData.append('state', JSON.stringify(state));
formData.append('command', 'add all details from dataset to invoice');

const response = await fetch('http://localhost:8000/api/bill/process', {
  method: 'POST',
  body: formData
});

const { state: updatedState, message } = await response.json();
console.log(message); // "✓ Agentic operation complete: ADD"
console.log(`Extracted ${updatedState.freight_bill.runs.length} runs`);
```

Step 3: Fill Empty LR Numbers

```
const formData = new FormData();
formData.append('state', JSON.stringify(updatedState));
formData.append('command', 'fill all empty LR No with values 40, 42, 44, 46');

const response = await fetch('http://localhost:8000/api/bill/process', {
  method: 'POST',
  body: formData
});

const { state: finalState } = await response.json();
```

Step 4: Export to PDF

```
const response = await fetch('http://localhost:8000/api/bill/export', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    state: finalState,
    format: 'pdf'
  })
});

const blob = await response.blob();
const url = URL.createObjectURL(blob);
window.open(url); // Open PDF in new tab
```

Complete Workflow: Tax Invoice

Step 1: Get Default State

```
const response = await fetch('http://localhost:8000/api/tax/default');
const { state } = await response.json();
```

Step 2: Set Invoice Details

```
const formData = new FormData();
formData.append('state', JSON.stringify(state));
formData.append('command', 'set invoice number to INV-2025-001 and invoice date to 24-11-2025');

const response = await fetch('http://localhost:8000/api/tax/process', {
  method: 'POST',
  body: formData
});

const { state: updatedState } = await response.json();
```

Step 3: Add Items from File

```
const formData = new FormData();
formData.append('file', fileInput.files[0]);
formData.append('state', JSON.stringify(updatedState));
formData.append('command', 'add all details from dataset');

const response = await fetch('http://localhost:8000/api/tax/process', {
  method: 'POST',
  body: formData
});

const { state: finalState } = await response.json();
```

Step 4: Export to HTML

```
const response = await fetch('http://localhost:8000/api/tax/export', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    state: finalState,
    format: 'html'
  })
});

const { html } = await response.json();
document.getElementById('preview').innerHTML = html;
```

Testing

Health Check

```
curl http://localhost:8000/health
```

Response:

```
{
  "status": "healthy",
  "timestamp": "2025-11-24T17:00:00.000000",
  "agents": {
    "tax_agent": "available",
    "bill_agent": "available"
  }
}
```

Interactive API Documentation

Visit `http://localhost:8000/docs` for interactive Swagger UI documentation where you can test all endpoints directly in your browser.

Best Practices

1. State Management

- **Always send the complete state** with each request
- **Store state in your frontend** (React state, Redux, etc.)
- **Don't rely on server-side sessions** - the API is stateless

2. File Uploads

- **Supported formats:** `.xlsx`, `.xls`, `.csv`
- **Max file size:** No hard limit, but keep files under 10MB for best performance
- **Column detection:** The AI automatically detects columns, but you can provide hints in the command

3. Commands

- **Be specific:** "add all rows" is better than "add data"
- **Use natural language:** "fill empty LR numbers with 40, 42, 44" works perfectly
- **Combine operations:** "set invoice number to INV-001 and date to 24-11-2025"

4. Error Handling

```
try {
  const response = await fetch('/api/bill/process', {
    method: 'POST',
    body: formData
  });

  if (!response.ok) {
    const error = await response.json();
    console.error('API Error:', error.detail);
    // Show error to user
  }

  const { state, message } = await response.json();
  // Success - update UI
} catch (error) {
  console.error('Network error:', error);
  // Handle network errors
}
```

Support

For questions or issues, contact the backend team or check the main documentation.

API Version: 1.0.0

Last Updated: November 24, 2025